

Homework 1: Simulating Simple LLM Inferences

Name: DEEP HIREN KOTECHA

PSU ID: djk6507

How to run the input generation script

The PDF asks for a brief description of how to run the script that generated the input CSVs.

Answer:

The input CSVs come from lab1/scripts/generate_input.py. It needs no extra dependencies beyond Python 3.10+. From the project root (or from Vidur-Agent/ after the lab1/ folder is in place):

python lab1/scripts/generate_input.py

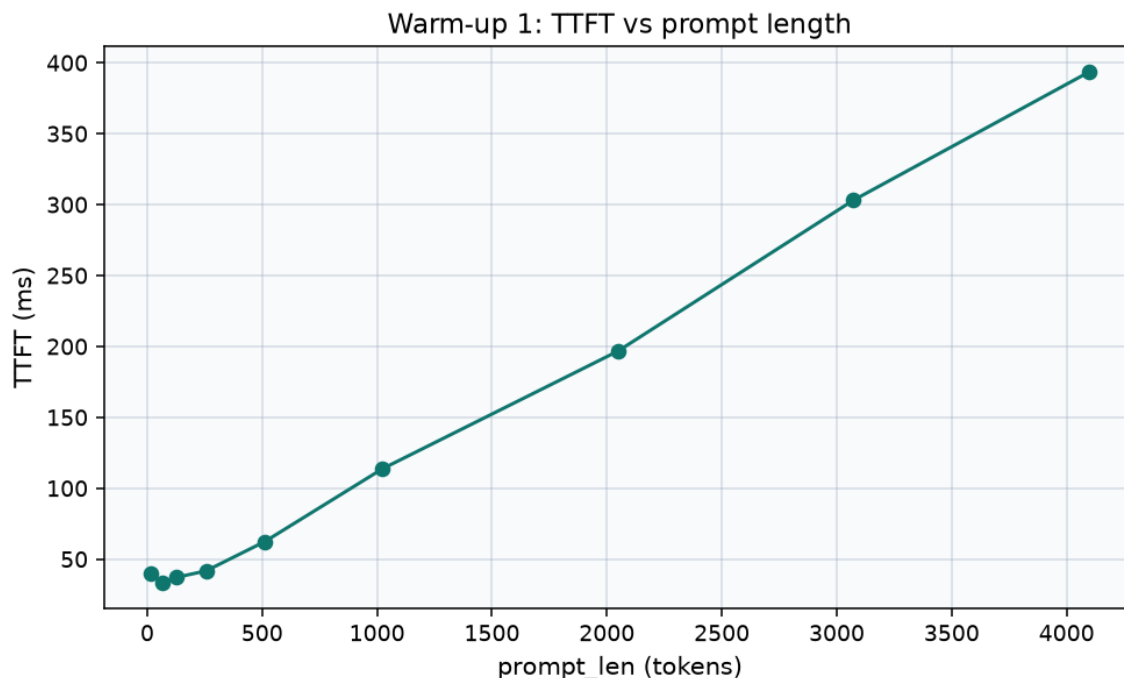
On W135, if you used uv for Vidur:

uv run python lab1/scripts/generate_input.py

Warm-up 1: Prompt length vs. latency (one request)

1. How does TTFT grow with prompt length? Why?

Figure W1.1 prompt_len vs TTFT

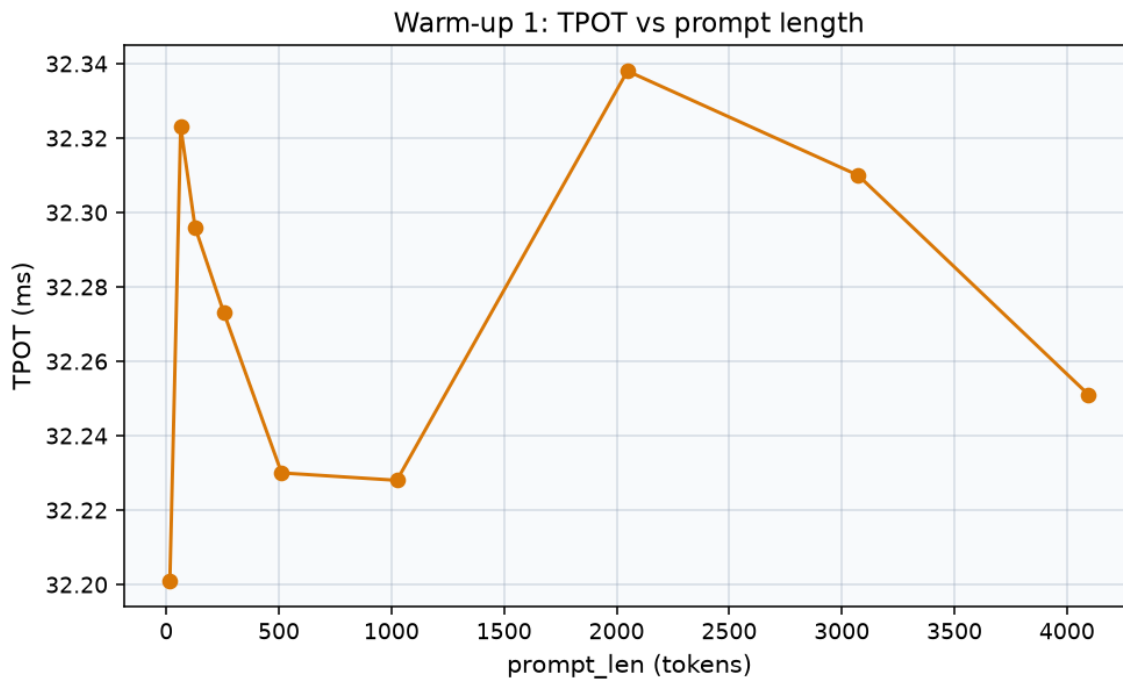


Answer:

As seen in the Plot above, it is evident that Prompt length is linearly related to TTFT. So TTFT increases with prompt length. This observation can be explained by the fact that in this experiment we use a **single request**. With one request there is no queue, so TTFT is only the wait until token 1, i.e. prefill. That prefill is one forward pass over n prompt tokens. Q/K/V, output projection, and the MLP are $O(n)$ and dominate on this 32B model. Attention is $O(n^2)$ but is negligible compared to those matmuls for $n \leq 4096$, so TTFT still tracks n almost linearly.

2. Does TPOT depend on prompt length? Why or why not?

Figure W1.2 *prompt_len vs TPOT*



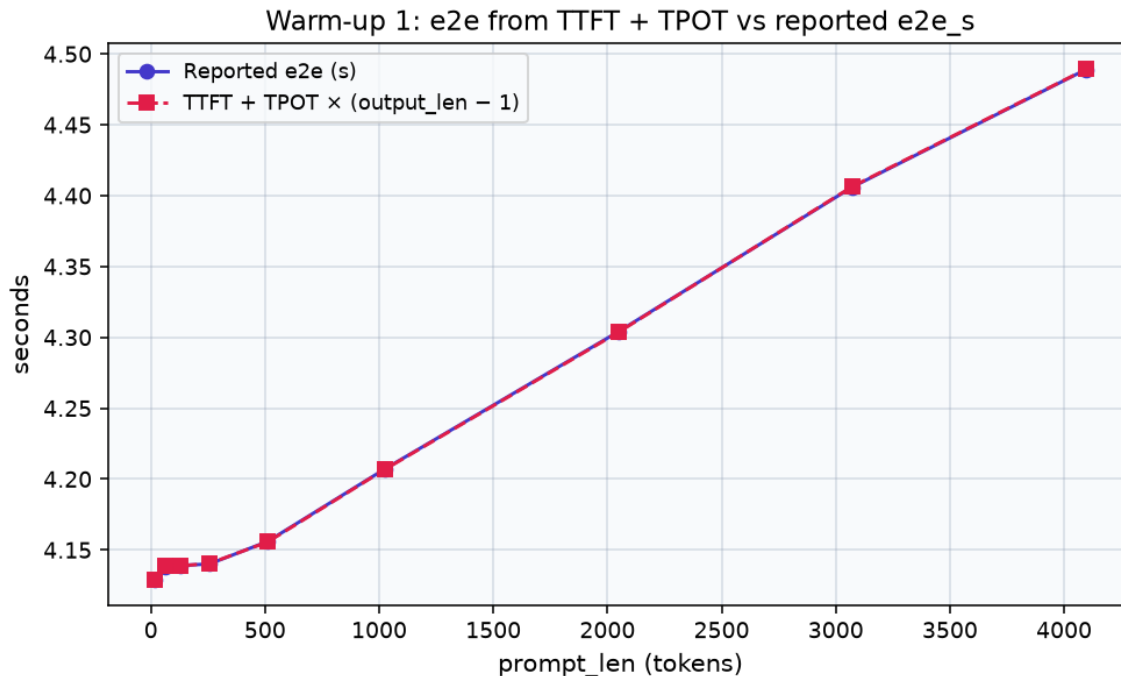
Answer:

TPOT is the average time of tokens after the first token. Those tokens are decoding steps, one new token at a time, with the KV cache. The prompt is no longer prefilled. So changing prompt length changes the one prefill (TTFT). It does not change the decode cost per request and per step. So that's why TTFT increases and TPOT has no discernible relation to prompt length. One other observation is that the range of TPOT values stays within 0.14ms, which is a negligible gap.

(Decode attention does see a longer KV as the prompt grows, so a tiny dependence is possible, but clearly it is not dominant to the metric)

3. From TTFT and TPOT, how can you calculate the end-to-end execution time? Does it match the reported e2e_s?

Figure W1.3 computed e2e vs reported e2e_s



Answer:

Yes. formula and e2e_s match on every row of Warm-up 1 (diff < 1ms). The two curves coincide in the plot. End-to-end time is first-token wait + rest of tokens.

$$e2e = TTFT + TPOT (output_len - 1)$$

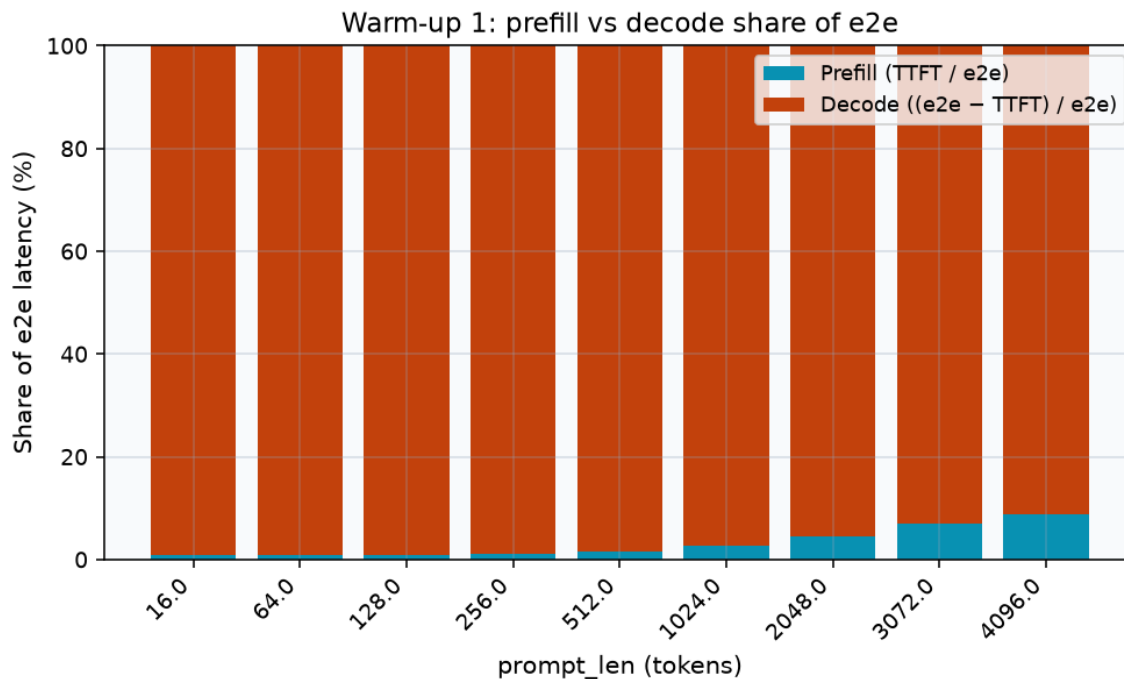
$$output_len = 128, \text{ so } e2e = TTFT + 127 * TPOT.$$

4. Percentage breakdown between prefill and decode latency for different prompt lengths. Between prefill and decode, which one is the bottleneck?

Answer:

The bars represent prefill share = $TTFT / e2e$ decode share = $(e2e - TTFT) / e2e$ Prefill is the prompt, decode is the other 127 tokens. As the prompt increases, the size of prefills slice increases (roughly 1% at 16 tokens $\rightarrow$ roughly 9% at 4096). Decodes share is slightly smaller, but still huge (about 99% $\rightarrow$ about 91%). **Decode is the bottleneck**, for each prompt length tested. Since output_len is fixed to 128, decode is still about 4.1 sec. Only Prefill goes from about 40ms to about 393ms, so it never catches up to decode.

Figure W1.4 prefill vs decode share of e2e



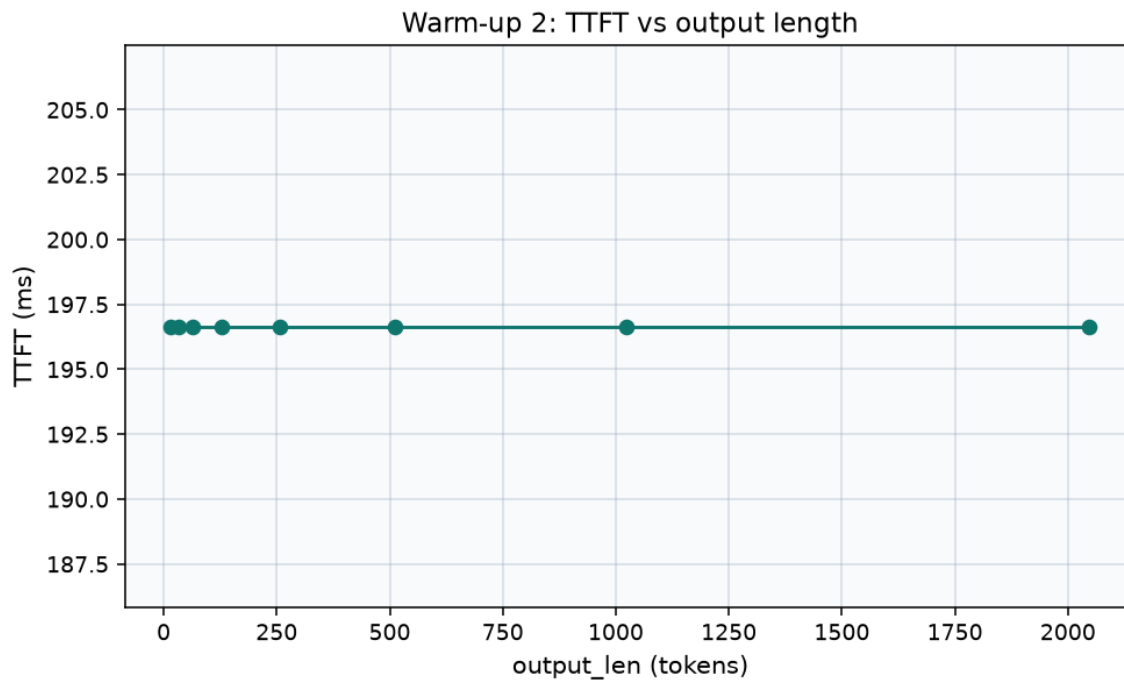
Warm-up 2: Output length vs. latency (one request)

1. Does TTFT grow with output length? Why or why not?

Answer:

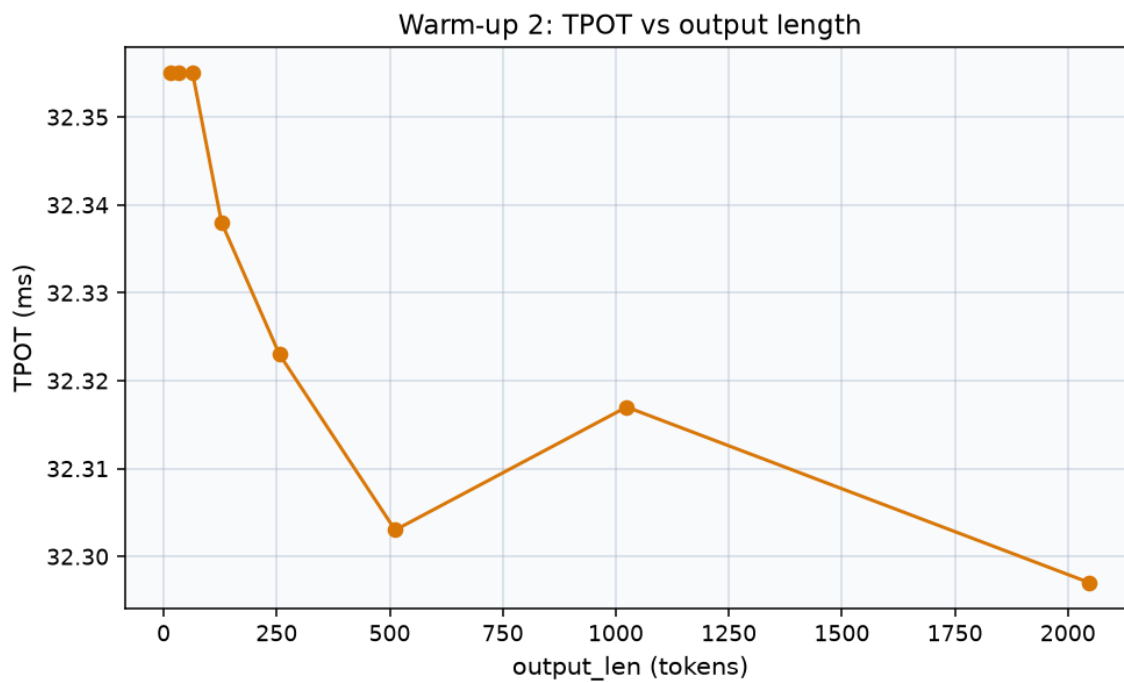
The plot shows that TTFT doesn't increase with output_len. Every run (output 16 through 2048) it is 196.654ms. It is a flat line. TTFT is time to token 1. That token is from prefill of the prompt. Here prompt_len is always 2048 so the prefill is always the same job. The output length doesn't matter because it is total tokens you generate. There are 2...N Decode tokens after the 1st token. They do not belong to TTFT as a metric, TTFT is limited to first token generation.

Figure W2.1 TTFT vs output_len



2. How does TPOT depend on output length? Why?

Figure W2.2 output_len vs TPOT

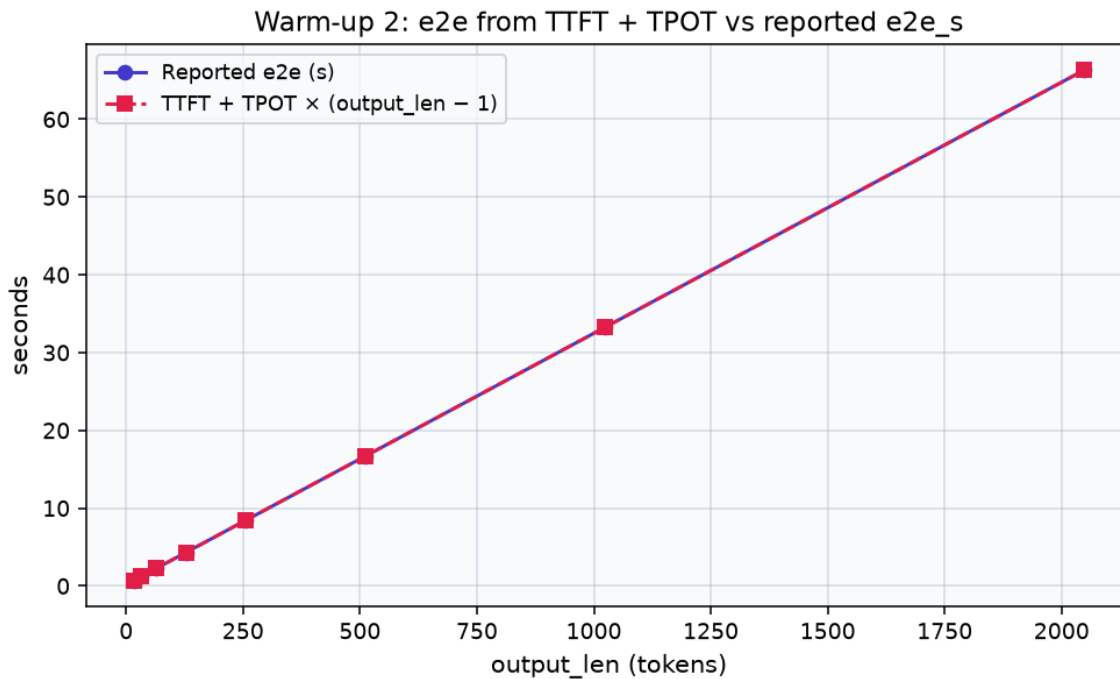


Answer:

The plot shows TPOT is not sensitive to output_len. Still about 32.3ms from output 16 to 2048. The swing is only about 0.06ms. The plot is zoomed in, so the noise looks like a dip. It remains flat because TPOT averages tokens after the first. Those are all still decode steps, one token, updating KV cache, same single request. Extending the output just means more of the same steps. It does not mean that each step is a different kind of work.

3. From TTFT and TPOT, how can you calculate the end-to-end execution time? Does it match the reported e2e_s?

Figure W2.3 computed e2e vs reported e2e_s



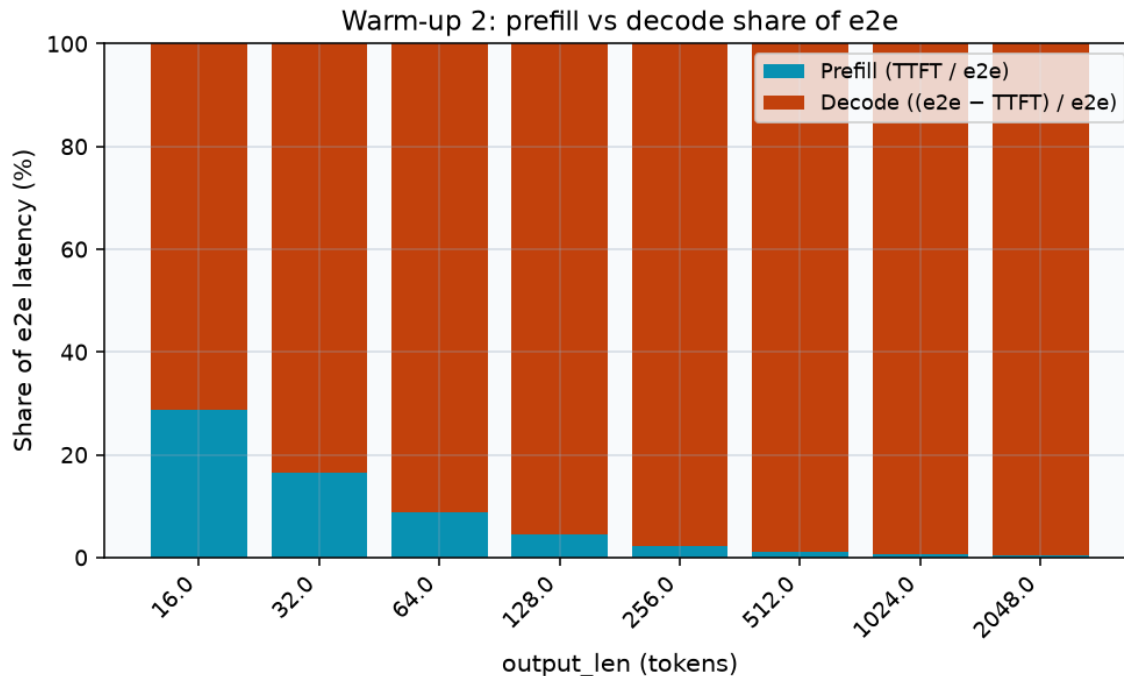
Answer:

Yes. Computed e2e and e2e_s overlap on plot for each output length (error < 1ms). Same identity as Warm-up 1 but now output_len is changing.

$e2e = TTFT + TPOT \times (output_len - 1)$ Token 1 is TTFT and all other tokens are TPOT. For this run TTFT is 196.654ms. output_len is variable, so e2e should almost linearly scale with output length (more decode tokens, same prefill).

4. Percentage breakdown between prefill and decode latency for different output lengths. Between prefill and decode, which one is the bottleneck?

Figure W2.4 prefill vs decode share of e2e



Answer:

The bars in the plot represent prefill share = $TTFT / e2e$. decode share = $(e2e - TTFT) / e2e$. Prefill fixed here (prompt 2048, TTFT 196.7ms). Decode increases with output_len. When the output length grows prefill's slice shrinks from about 29% at output 16 to about 0.3% at output 2048. Decode 71% => 99.7%. Decode is the bottleneck for all the output lengths. Even at output 16 decode is still the bigger piece. For long outputs, it is almost the entire e2e.

Experiment 1: Batch size vs. latency

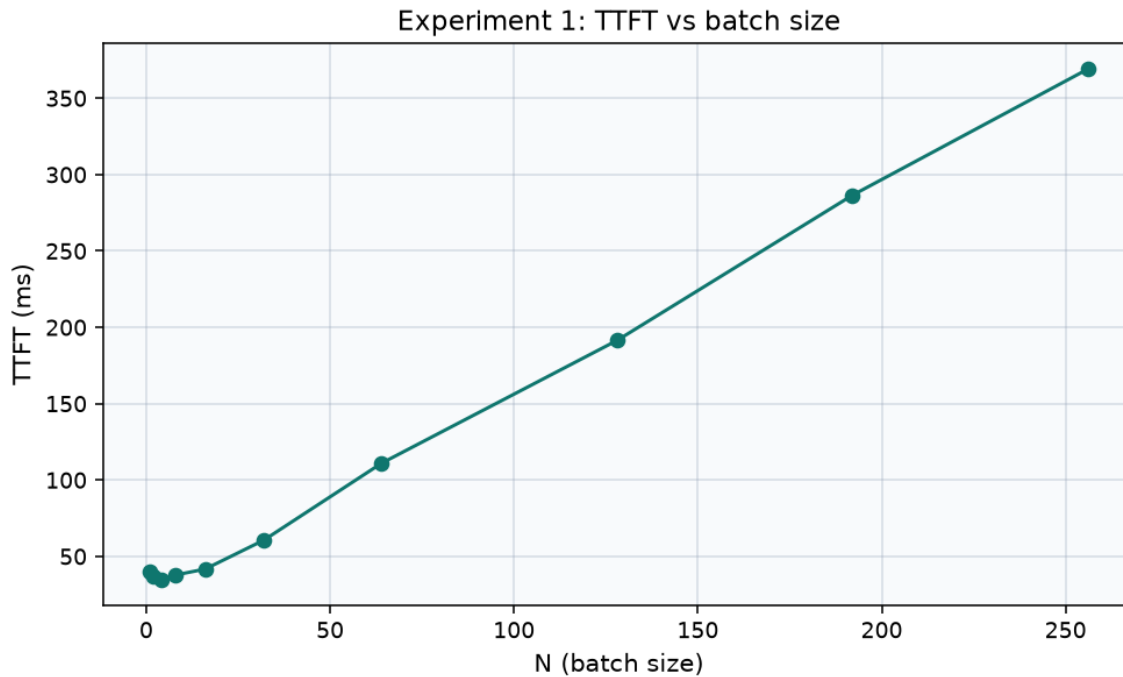
N identical requests all arrive at time 0 (count). prompt_len=16, output_len fixed; N from 1 to 256.

1. How does TTFT change with N? Why?

Answer:

In this experiment we change only N. Each request still has prompt_len=16, output_len=128, arrival 0. The plot shows TTFT still increasing with N (40ms at N=1 → 369ms at N=256). TTFT goes up because all N short prompts prefill in one batch. This step does $N * 16$ tokens (16 when N = 1, 4096 when N = 256). A bigger batched prefill takes more time, so everyone's first token waits longer.

Figure E1.1 N vs TTFT



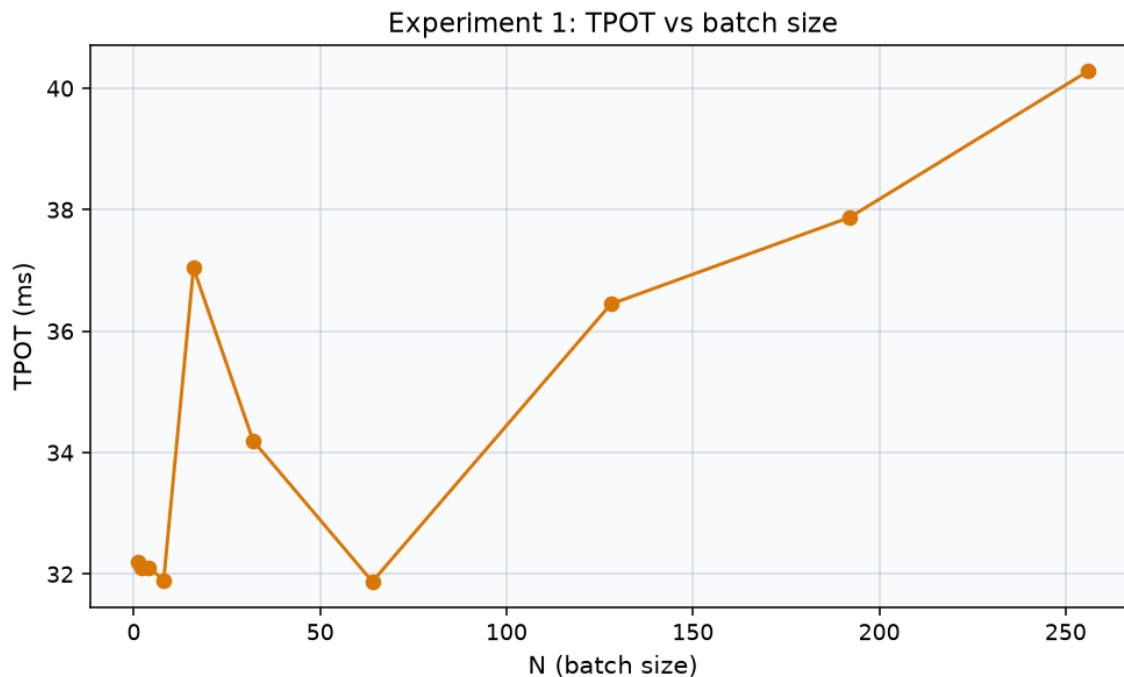
2. How does TPOT change with N? Why?

Answer:

The Plot shows that TPOT has not been greatly influenced by the increase in batch size, with the execution time only increasing from 32ms in (N=1) to around 40ms even when the batch size increases from 1 to 256. This is expected behavior from a decoding process that is memory-bandwidth-bound, since model weights are loaded at every step regardless of the number of requests. Therefore, increase in (N) does not result in the increase in execution time in proportion. On the other hand, throughput surprisingly increases from around 31 tokens/s to more than 6300 tokens/s. Higher batch sizes allow more tokens to be processed simultaneously during the matrix multiplication operations, but they also cause some increase in execution time.

At the same time, there are some unexpected things in the data. At (N=16), decoding stages have uncharacteristic performance, whereas neighboring batch sizes, (N=8) and (N=32), have better results. This could mean that maybe spike is simply a wrong prediction for the batch size rather than real change in performance. The same inconsistency is observed at (N=64).

Figure E1.2 N vs TPOT



Experiment 2: New requests arrive!

4a. One request alone (baseline)

Answer:

The first request at $t=0$ is given, with prompt 1024 and produce 512 (like request 0 in 4b–4d).. There is 0 waiting time on queue. The TTFT is 113.7ms based on one prefill step of length of 1024 tokens. The TPOT is 32.26ms. The e2e time is 16.6 seconds. The typical time gap is in the range of 32.2–32.3ms, with the highest gap recorded at 32.31ms. Since there are no other processes running, nothing interferes with the decoding step.

4b. Add a second request that arrives while the first is still generating output. Compare the first request's TPOT and token gaps with 4a. Why did they change? Use tokens.csv and steps.csv to exactly pinpoint what happened.

Answer:

The second request makes an entry at $t=1.0$ seconds (prompt size being 1024 and output size being 64) while request zero is being decoded. The second request must wait for 15.5ms for the currently running step to end. The scheduler would put request one's entire 1024 tokens in the same step as request zero's next decode (steps.csv step 29: start 1.0155 s, end 1.1326 s, duration 117.07 ms, prefill_tokens=1024, decode_tokens=1, requests 0 and 1; 117.07 ms is the

duration of this step, not the wall-clock time the token was produced). Step 29 takes 117.07 ms. tokens.csv shows request 0, token 30: time_s =1.133 s, gap_ms=117.07, step=29, so request 0's gap for that token is 117.07 ms as opposed to about 32 ms in 4a. Therefore, request zero has a worst-case scenario gap as compared to the earlier 32.3ms measured before. Now each step will have decoding done meaning one token will be decoded at approx 32.1ms, leading to very little change in steady – state decoding time overall since the time would only extend from 32.26 to 32.42ms later on leading to an increase only in gaps.

Figure E2.1 request 0 token gaps, 4a vs 4b

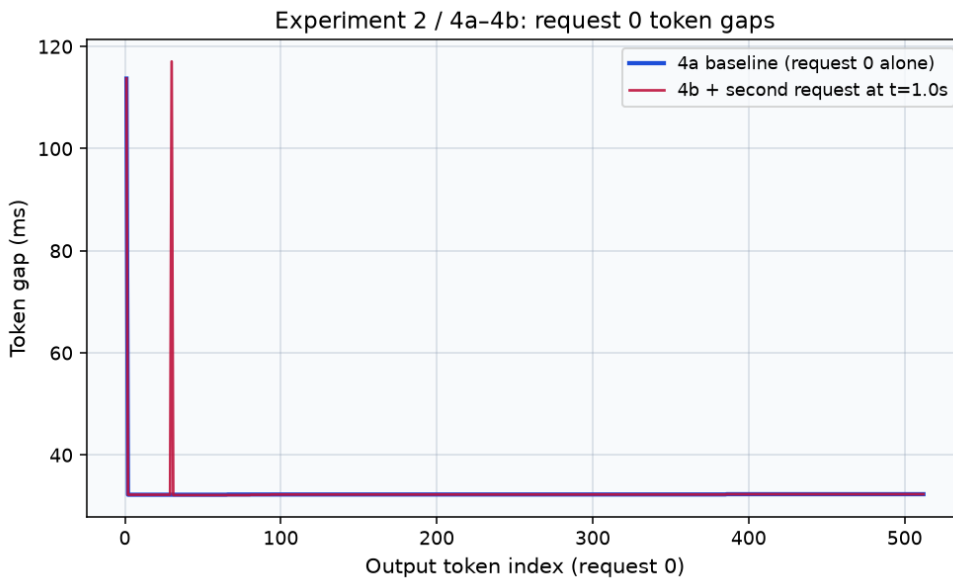
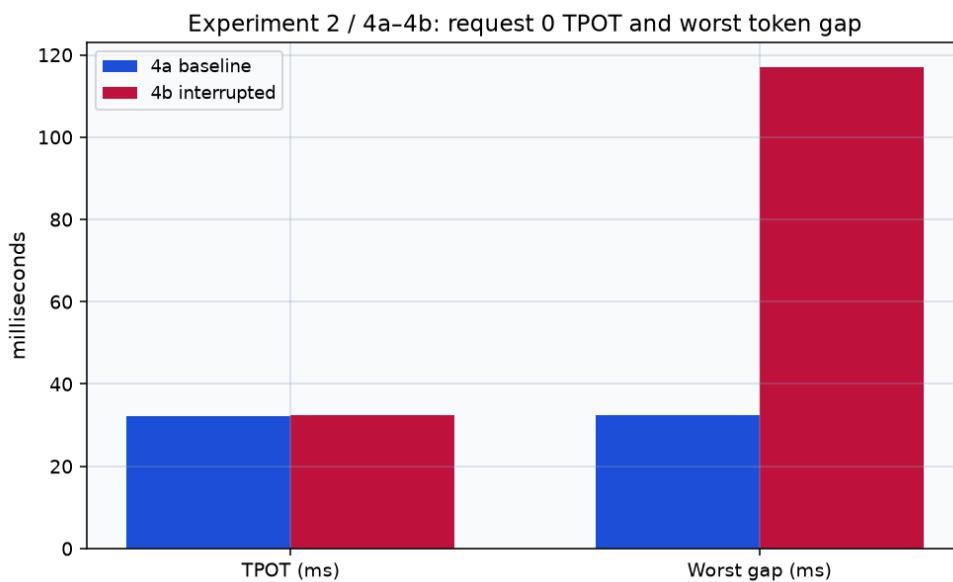
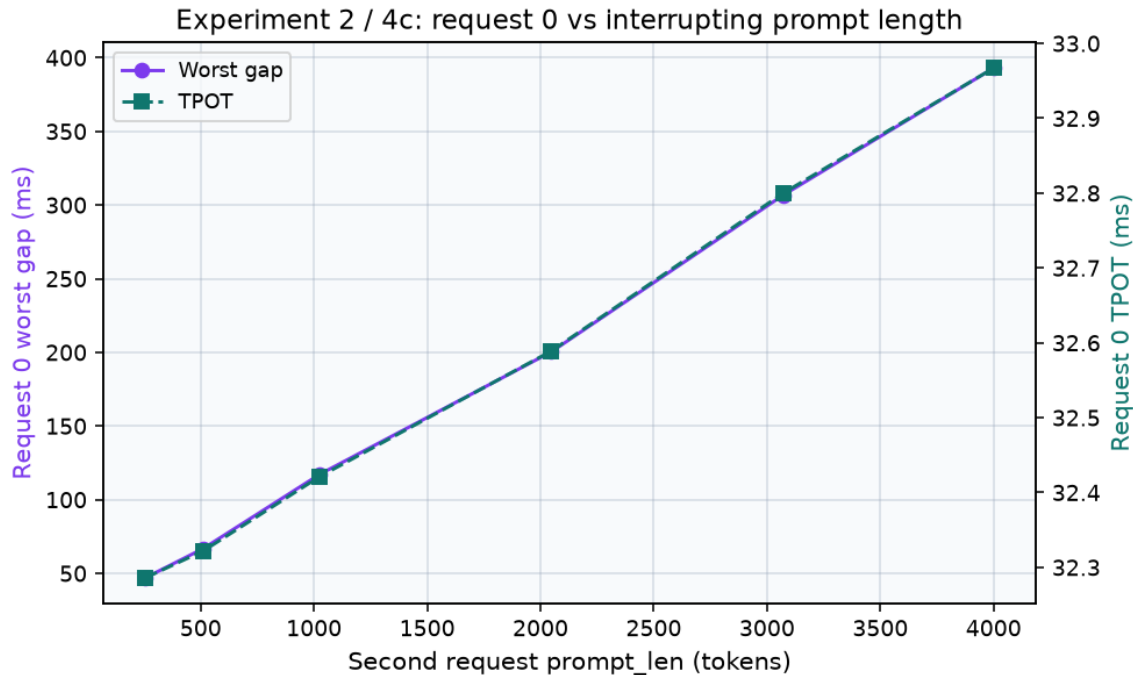


Figure E2.2 request 0 TPOT and worst gap, 4a vs 4b



4c. Vary the second request's prompt length (few hundred tokens up to 4000). For each, discuss how the first request's worst token gap, and its TPOT, changes.

Figure E2.3 request 0 worst gap and TPOT vs second prompt



Answer:

Request 0 remains unchanged (prompt 1024, result 512, $t=0$). second request comes in at $t=1.0$ s with output 64. only changing its prompt length: 256, 512, 1024, 2048, 3072, 4000. request 0's worst gap is increasing: $47 \rightarrow 67 \rightarrow 117 \rightarrow 200 \rightarrow 307 \rightarrow 393$ ms. the gap always occurs on token 30, the first token after the second request arrives.

The chunk size being 4096 means that all second's prompt till 4000 would fit into one chunk. the scheduler puts all that prefill into the same step as next decode for request 0. request 0's token gets delayed for that whole step. therefore, the worst gap would be approximately the time taken to prefill the second prompt. this time closely resembles the Warm-up 1 TTFT of the same prompt length (1024 $\rightarrow$ 114 vs 117ms, 2048 $\rightarrow$ 197 vs 200ms, 4096 $\rightarrow$ 393ms vs 393ms for 4000 prompt length). this suggests that the worst gap takes on similar approximate time to the prefill growth.

TPOT goes only $32.29 \rightarrow 32.37$ ms for the whole sweep count. the delay occurs at one step while all other decode process happens in about 32ms. TPOT does an average of that one delay over 511 gaps (at 4000: about 361ms extra / 511 = 0.7ms). therefore, TPOT doesn't showcase the situation of request 0.

4d. Repeat 4c at several chunk sizes (4096, 2048, 1024, 512, 256). Discuss what happens.

Figure E2.4 request 0 worst gap vs second prompt, by chunk size

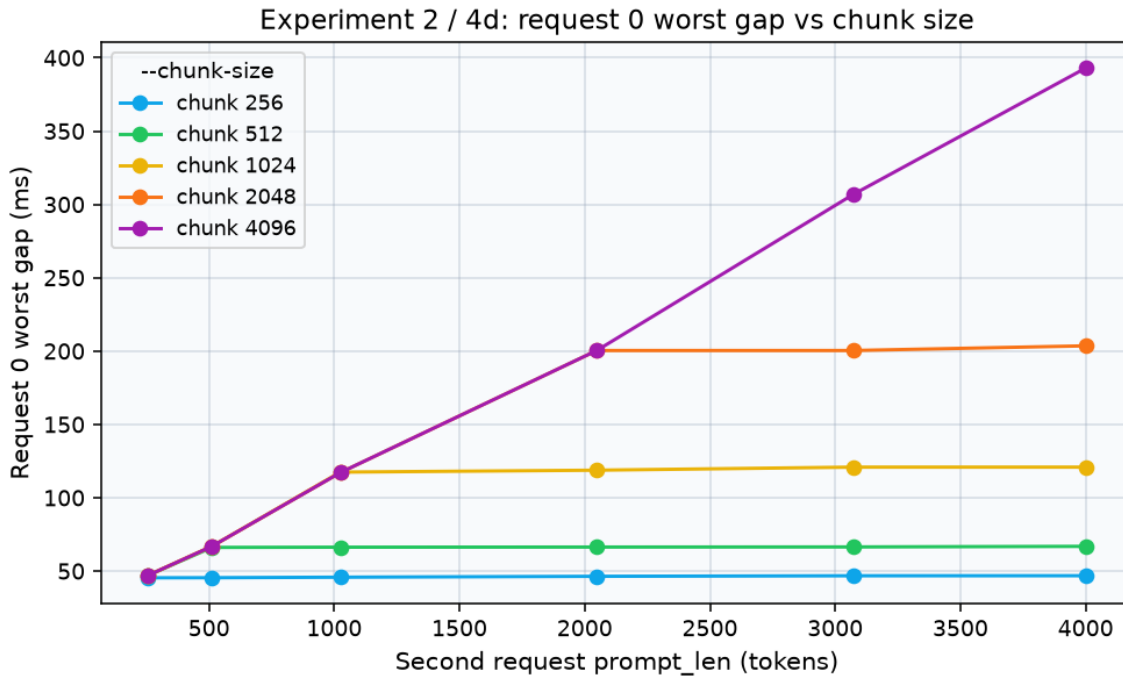


Figure E2.5 request 0 TPOT vs second prompt, by chunk size



Answer:

We repeat experiment 4c with chunk sizes of 4096, 2048, 1024, 512, and 256. Each step has a maximum chunk size, subtracting the number of tokens that are decoded in that step, and the second prompt is divided into several steps, so with 4000 tokens and a chunk of 256, we have a total of 16 steps of 255 tokens. Request 0 receives a token in every step, meaning that its worst-case scenario is equal to prefill less than the size of the chunk, which amounts to approximately 46, 66, 120, 200, and 393ms depending on whether we use chunk size of 256 or 4096. When the length of the second prompt exceeds the size of the chunk, the curve becomes flat.

Every chunking segment also produces the next token of request 0, which would have been taken 32ms to decode. Therefore, there is only extra delay of over 32ms from every segment of chunking. At chunk 256, this extra delay is about 222ms ($16 \times 14\text{ms}$ approx), and at chunk 4096, it is around 361ms. These correspond to TPOT 32.69ms vs 32.97ms over 511 gaps. Still, chunking prolongs the second prefill overall to around 737ms vs 393ms.